VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Shraddha Sinha	24BHI10038
Slot	**Date of Submission**
A24	30-03-26

1. Introduction

Open source software is all about sharing. Its code is out in the open for anyone to see, improve, and build on—which is why it's become the backbone of so much modern technology. From the websites you visit to the apps powering artificial intelligence, open source is everywhere. It brings people together from around the world to collaborate, spark new ideas, and keep making things better.

This project is a deep dive into one of the most popular open-source tools out there: Python. Known for its simple, readable code, Python has taken the programming world by storm. Here, we'll explore what makes Python tick—its origins, its licensing, and why it matters so much in the tech world. By looking at Python's story, we get a front-row seat to how open-source projects grow, thrive, and change the way we use technology.

But this isn't just theory. You'll also get hands-on practice with Linux and basic shell scripting, learning how to install, manage, and use Python in real-life situations. This mix of understanding and doing makes the learning experience not just informative, but genuinely useful.

2. About Python

Python is a high-level, interpreted programming language used for a wide range of applications, including web development, data science, artificial intelligence, and automation. It was created by Guido van Rossum and released in 1991 with the goal of making programming simple and easy to understand. Python emphasizes code readability and uses a clear and straightforward syntax, making it beginner-friendly.

One of the key features of Python is its simplicity and versatility. Developers can write fewer lines of code compared to other languages, which increases productivity. Python supports multiple programming paradigms such as procedural, object-oriented, and functional programming. It also has a vast standard library and a large ecosystem of third-party packages, which makes it suitable for various tasks.

Python is widely used in the industry and is considered one of the most popular programming languages today. It is the backbone of many modern technologies, especially in fields like machine learning, data analysis, and automation. Platforms like web frameworks, data tools, and AI libraries rely heavily on Python. Due to its flexibility, strong community support, and continuous development, Python has become an essential tool for developers across different domains.

3.Part A — Origin and Philosophy

Python, today, is a high-level, interpreted programming language which is known for its easy and simple syntax, readability and versatility. Python nowadays can be seen being used everywhere from beginners learning how to code a simple print statement to high end developers using it for web apps, automation tools, data science and artificial intelligence. But

when was it created? Who created it? And what was the intention behind creating it? Let's take a dive into its origin:

While the word "python" might bring to mind a large snake, the language's name comes from the classic BBC comedy series Monty Python's Flying Circus. Programming language python was conceived in the late 1980s and was later implemented by Guido Van Rossum at CWI in Netherlands in 1991. Van Rossum was Python's principal author and had a central role in deciding the direction of Python until he stepped down as leader. Python began as the vision and work of a single person. Unlike most languages born in big tech companies, Python was created by Guido, and he was driven by a single goal: to make programming more intuitive and enjoyable. From the simple idea of a man, a global community has grown. Thousands of people like developers, educators, scientists, and enthusiasts still continue to shape Python, expanding its reach into AI, data science, education, and beyond to many more areas.

Python is open source and not kept proprietary because primary goal of the creator Guido Van Rossum was to create a tool for productivity and ease of use, not to make money, and he recognized that involving a community was the best way to develop a robust language.

The creator had a simple but visionary set of goals or his language. He wanted the language to be easy and intuitive, open source, readable and elegant, and practical and efficient.

At the time even when other programming languages existed, they prioritized performance (C/C++) or were designed strictly for teaching, but python gained popularity because it aimed for programmer productivity and ease of use.

A2. The license — what it actually says

Python software and documentation are licensed under the Python Software Foundation License Version 2.

What are the four freedoms of free software? Does your software's license grant all four?

The four freedoms of free software are as follows:

- The freedom to run the program for any purpose
- The freedom to study the program works by accessing its source code
- the freedom to modify the software according to one's needs
- the freedom to distribute copies of the original or modified software

Python grants all the above mentioned four freedoms making it truly open and accessible for all.

Can a company take this software, modify it, and sell it without sharing their changes? Why or why not?

A company can use Python for commercial purposes and can even modify the software and build products using it without being obliged to share the changes publicly, unlike other licenses such as GPL(general public license) which also requires to share modified versions of open source.

What is the difference between GPL and MIT licensing? Which would you choose for something you built, and why?

GPL is also known as “copyleft” license means any modifications must be shared openly, meanwhile MIT License is more permissive and allows developers to use, modify, and distribute software with minimal restrictions.

If I had to make a choice, I would choose GPL as I would like to know what new can be created by something I created and would like to see the new ideations and modifications.

Is there a difference between 'free as in free beer' and 'free as in freedom'? Give an example from your chosen software.

Free beer refers to software being available at no cost, while free freedom means users have the right to modify and control the software. Python allows both, it is free to use, and it also gives users full freedom to explore, modify, and share it.

A3. The ethics of open source

Should all software be open source? Make an argument for and against

Open source promotes transparency, as anyone can view and verify the code and use the code to build their ideas which helps in learning and innovation but not all software should be open source as it takes a significant amount of time, efforts and resources of a person or a company to build a software and making it open source without any returns is not fair for the developing unit.

Is it ethical for a company to build a profitable product on free community labor?

Companies use open-source code, which is created and modified by contributors or volunteers, for the development of their projects and earn all the profits themselves but the contributors get no financial compensation for it which does not seem fair but the contributors or volunteers already knew beore contributing to open source that their code can be used but they still voluntarily did that so in my opinion i think it is ethical.

What does “standing on the shoulders of giants” mean in software? Does open source enable or undermine innovation?

Meaning of the phrase “standing on the shoulders of giants” is building new ideas and technologies based on the work of previous developers. In software, this is really applicable as developers can access existing code and use it as a foundation for further innovation.

Open source enables innovation because it allows developers to avoid reanalyzing basic components and instead focus on improving and innovating. It fastens up the development. Many modern technologies, including artificial intelligence and web development, have advanced rapidly due to open-source contributions.

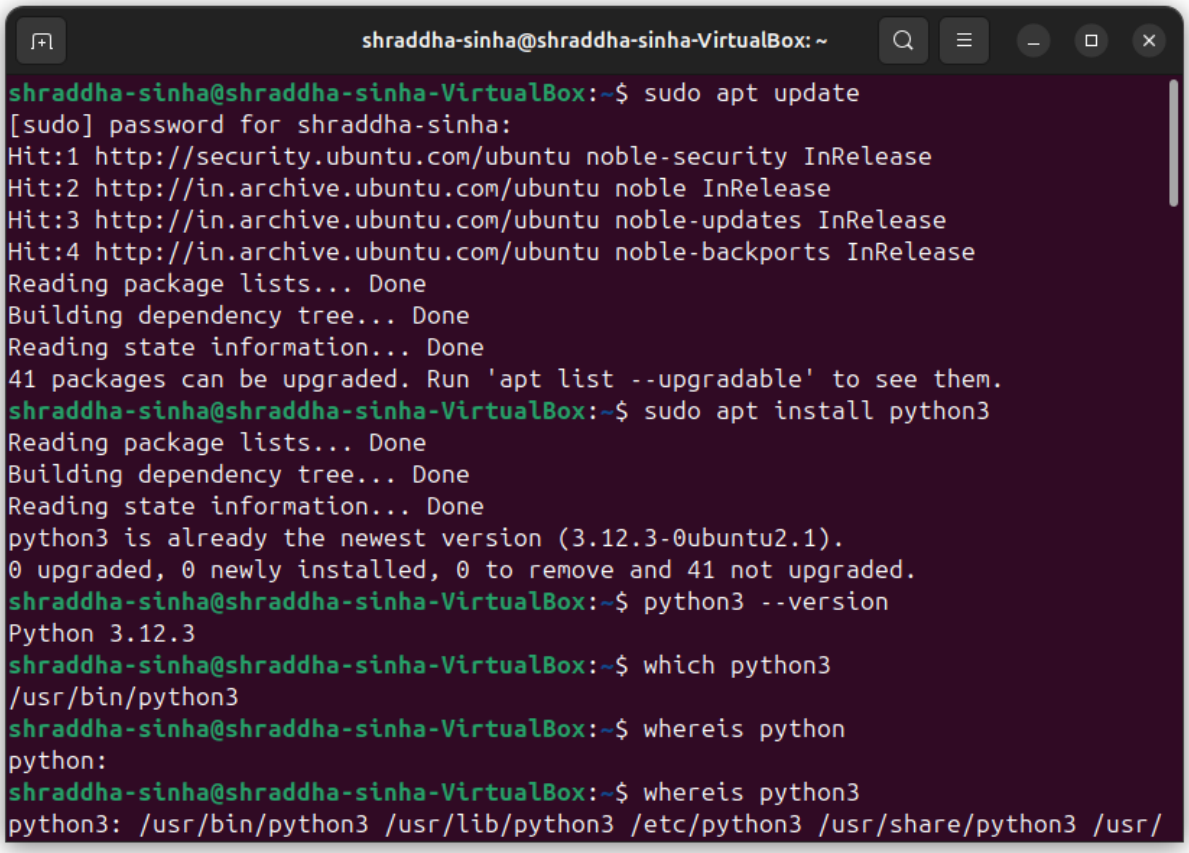
However, too much dependence on existing open-source solutions may reduce originality and creativity. Some developers might rely heavily on available libraries instead of developing new ideas from scratch.

Overall, open source enables innovation more than it undermines it. It provides a strong base for creativity while still allowing developers to build unique and original solution.

Part B — Linux Footprint (Unit 2)

1. Installation Method

Python was installed on the Linux system using the APT package manager, which is the default package management tool in Ubuntu. The command sudo apt update was first used to update the package list, followed by sudo apt install python3 to install Python. The system showed that Python was already installed with the latest version, which confirms that Python is available through the official Ubuntu repositories. This means Python can be easily installed and managed without compiling from source.



2.Key Directories:

- **/usr/bin/python3**: This is where the Python executable lives. You can run Python from here system-wide.
- **/usr/lib/python3**: Holds Python libraries that your code and other programs may use.
- **/etc/python3**: Contains configuration files related to Python.
- **/usr/share/python3**: Stores shared resources and documentation for Python.
- **/var/log**: While not specific to Python, this directory contains system logs—handy for troubleshooting and monitoring.

Linux keeps things organized by putting software and related files in these clearly defined locations, making management and maintenance much easier.

```
shraddha-sinha@shraddha-sinha-VirtualBox: ~
shraddha-sinha@shraddha-sinha-VirtualBox:~$ sudo apt update
[sudo] password for shraddha-sinha:
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://in.archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
41 packages can be upgraded. Run 'apt list --upgradable' to see them.
shraddha-sinha@shraddha-sinha-VirtualBox:~$ sudo apt install python3
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3 is already the newest version (3.12.3-0ubuntu2.1).
0 upgraded, 0 newly installed, 0 to remove and 41 not upgraded.
shraddha-sinha@shraddha-sinha-VirtualBox:~$ python3 --version
Python 3.12.3
shraddha-sinha@shraddha-sinha-VirtualBox:~$ which python3
/usr/bin/python3
shraddha-sinha@shraddha-sinha-VirtualBox:~$ whereis python
python:
shraddha-sinha@shraddha-sinha-VirtualBox:~$ whereis python3
python3: /usr/bin/python3 /usr/lib/python3 /etc/python3 /usr/share/python3 /usr/share/man/man1/python3.1.gz
shraddha-sinha@shraddha-sinha-VirtualBox:~$ uname -a
Linux shraddha-sinha-VirtualBox 6.17.0-19-generic #19~24.04.2-Ubuntu SMP PREEMPT_DYNAMIC Fri Mar  6 23:08:46 UTC 2 x8
6_64 x86_64 x86_64 GNU/Linux
shraddha-sinha@shraddha-sinha-VirtualBox:~$ whoami
shraddha-sinha
shraddha-sinha@shraddha-sinha-VirtualBox:~$ apt list --upgradable
Listing... Done
alsa-ucm-conf/noble-updates 1.2.10-1ubuntu5.10 all [upgradable from: 1.2.10-1ubuntu5.9]
```

3. User and group

When you run Python, it operates under your current user account—in this case, that's:

shraddha-sinha

But installing Python required special permissions. That's why you used the sudo command, which temporarily gives you root (administrator) privileges. This separation helps keep your system safe: everyday users can't accidentally mess up important files, and only admins can install or update software.


```
shraddha-sinha@shraddha-sinha-VirtualBox: ~
shraddha-sinha@shraddha-sinha-VirtualBox:~$ sudo apt update
[sudo] password for shraddha-sinha:
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://in.archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
41 packages can be upgraded. Run 'apt list --upgradable' to see them.
shraddha-sinha@shraddha-sinha-VirtualBox:~$ sudo apt install python3
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3 is already the newest version (3.12.3-0ubuntu2.1).
0 upgraded, 0 newly installed, 0 to remove and 41 not upgraded.
shraddha-sinha@shraddha-sinha-VirtualBox:~$ python3 --version
Python 3.12.3
shraddha-sinha@shraddha-sinha-VirtualBox:~$ which python3
/usr/bin/python3
shraddha-sinha@shraddha-sinha-VirtualBox:~$ whereis python
python:
shraddha-sinha@shraddha-sinha-VirtualBox:~$ whereis python3
python3: /usr/bin/python3 /usr/lib/python3 /etc/python3 /usr/share/python3 /usr/share/man/man1/python3.1.gz
shraddha-sinha@shraddha-sinha-VirtualBox:~$ uname -a
Linux shraddha-sinha-VirtualBox 6.17.0-19-generic #19~24.04.2-Ubuntu SMP PREEMPT_DYNAMIC Fri Mar  6 23:08:46 UTC 2 x8
6_64 x86_64 x86_64 GNU/Linux
shraddha-sinha@shraddha-sinha-VirtualBox:~$ whoami
shraddha-sinha
shraddha-sinha@shraddha-sinha-VirtualBox:~$ apt list --upgradable
Listing... Done
alsa-ucm-conf/noble-updates 1.2.10-1ubuntu5.10 all [upgradable from: 1.2.10-1ubuntu5.9]
```

4.Service Management

Python isn't a background service itself—it's an interpreter that runs scripts. But, if you build a Python-powered service or app, you can control it using Linux's service manager (systemd) with commands like:

- systemctl start <service>
- systemctl stop <service>
- systemctl restart <service>

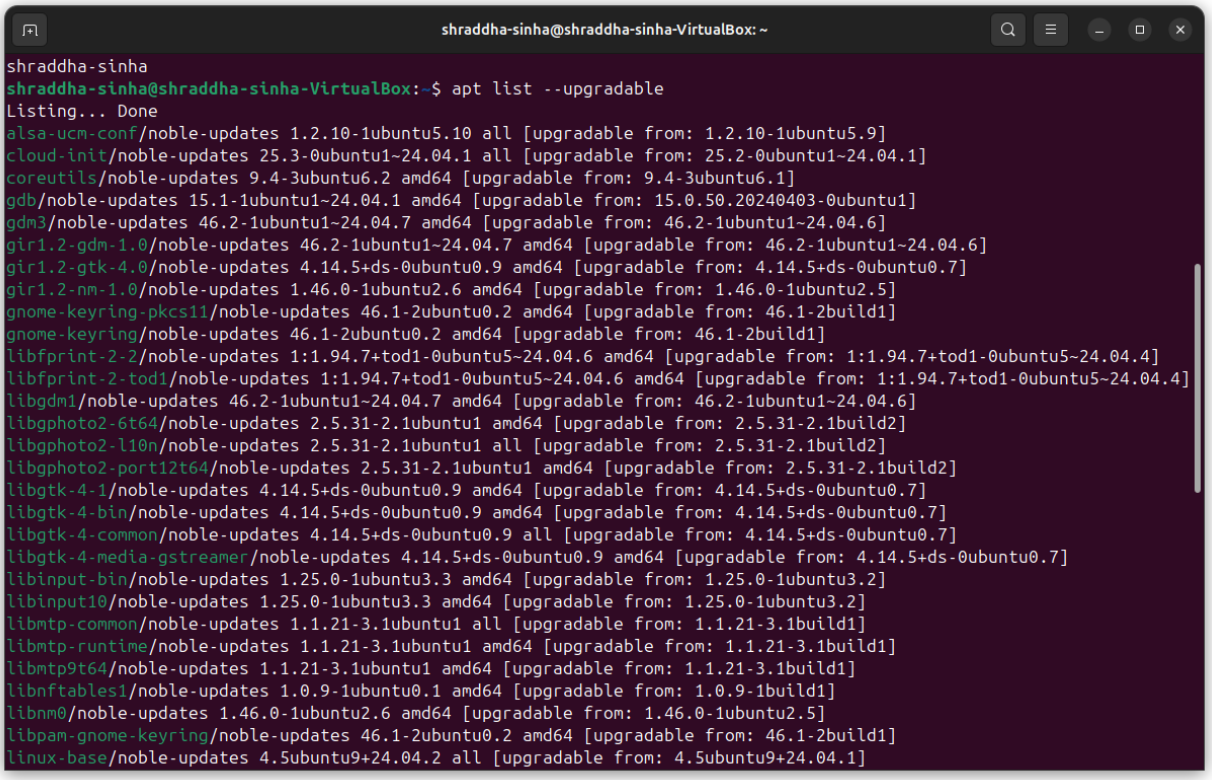
This is how Linux keeps services running smoothly and lets you manage them easily.

5. Update Model

Linux makes updating software simple and centralized with APT. You can check for updates using:

- apt list --upgradable (see what can be updated)
- sudo apt upgrade (actually install the updates)

Updates come from Ubuntu and the open-source community. They include security fixes, bug patches, and performance tweaks—helping keep your system secure and up-to-date without any hassle.



```
shraddha-sinha@shraddha-sinha-VirtualBox: ~
libinput10/noble-updates 1.25.0-1ubuntu3.3 amd64 [upgradable from: 1.25.0-1ubuntu3.2]
libmtp-common/noble-updates 1.1.21-3.1ubuntu1 all [upgradable from: 1.1.21-3.1build1]
libmtp-runtime/noble-updates 1.1.21-3.1ubuntu1 amd64 [upgradable from: 1.1.21-3.1build1]
libmtp9t64/noble-updates 1.1.21-3.1ubuntu1 amd64 [upgradable from: 1.1.21-3.1build1]
libnftables1/noble-updates 1.0.9-1ubuntu0.1 amd64 [upgradable from: 1.0.9-1build1]
libnm0/noble-updates 1.46.0-1ubuntu2.6 amd64 [upgradable from: 1.46.0-1ubuntu2.5]
libpam-gnome-keyring/noble-updates 46.1-2ubuntu0.2 amd64 [upgradable from: 46.1-2build1]
linux-base/noble-updates 4.5ubuntu9+24.04.2 all [upgradable from: 4.5ubuntu9+24.04.1]
network-manager-config-connectivity-ubuntu/noble-updates 1.46.0-1ubuntu2.6 all [upgradable from: 1.46.0-1ubuntu2.5]
network-manager-openvpn-gnome/noble-updates 1.10.2-4ubuntu0.2 amd64 [upgradable from: 1.10.2-4build2]
network-manager-openvpn/noble-updates 1.10.2-4ubuntu0.2 amd64 [upgradable from: 1.10.2-4build2]
network-manager/noble-updates 1.46.0-1ubuntu2.6 amd64 [upgradable from: 1.46.0-1ubuntu2.5]
nftables/noble-updates 1.0.9-1ubuntu0.1 amd64 [upgradable from: 1.0.9-1build1]
openvpn/noble-updates 2.6.19-0ubuntu0.24.04.1 amd64 [upgradable from: 2.6.14-0ubuntu0.24.04.3]
python3-software-properties/noble-updates 0.99.49.4 all [upgradable from: 0.99.49.3]
software-properties-common/noble-updates 0.99.49.4 all [upgradable from: 0.99.49.3]
software-properties-gtk/noble-updates 0.99.49.4 all [upgradable from: 0.99.49.3]
systemd-hwe-hwdb/noble-updates 255.1.7 all [upgradable from: 255.1.6]
ubuntu-drivers-common/noble-updates 1:0.9.7.6ubuntu3.6 amd64 [upgradable from: 1:0.9.7.6ubuntu3.5]
wpasupplicant/noble-updates 2:2.10-21ubuntu0.4 amd64 [upgradable from: 2:2.10-21ubuntu0.3]
shraddha-sinha@shraddha-sinha-VirtualBox:~$ sudo apt upgrade
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
Get more security updates through Ubuntu Pro with 'esm-apps' enabled:
  libzvbi-common libcjson1 libpostproc57 libmbedcrypto7t64 libavcodec60
  libgstreamer-plugins-bad1.0-0 libzvbi0t64 libavutil58 libswscale7
  libswresample4 libavformat60 libavfilter9
Learn more about Ubuntu Pro at https://ubuntu.com/pro
The following packages will be upgraded:
  alsa-ucm-conf cloud-init coreutils gdb gdm3 gir1.2-gdm-1.0 gir1.2-gtk-4.0
```

```
shraddha-sinha@shraddha-sinha-VirtualBox: ~
Setting up network-manager (1.46.0-1ubuntu2.6) ...
Setting up software-properties-common (0.99.49.4) ...
Setting up libgphoto2-6t64:amd64 (2.5.31-2.1ubuntu1) ...
Removing 'diversion of /lib/udev/hwdb.d/20-libgphoto2-6.hwdb to /lib/udev/hwdb.d/20-libgphoto2-6.hwdb.usr-is-merged b
y usr-is-merged'
Removing 'diversion of /lib/udev/rules.d/60-libgphoto2-6.rules to /lib/udev/rules.d/60-libgphoto2-6.rules.usr-is-merg
ed by usr-is-merged'
Setting up network-manager-openvpn (1.10.2-4ubuntu0.2) ...
Setting up software-properties-gtk (0.99.49.4) ...
Setting up network-manager-config-connectivity-ubuntu (1.46.0-1ubuntu2.6) ...
Processing triggers for install-info (7.1-3build2) ...
Processing triggers for desktop-file-utils (0.27-2build1) ...
Processing triggers for hicolor-icon-theme (0.17-2) ...
Processing triggers for gnome-menus (3.36.0-1.1ubuntu3) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
Processing triggers for rsyslog (8.2312.0-3ubuntu9.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libglib2.0-0t64:amd64 (2.80.0-6ubuntu3.8) ...
Processing triggers for dbus (1.14.10-4ubuntu4.1) ...
Processing triggers for shared-mime-info (2.4-4) ...
Processing triggers for udev (255.4-1ubuntu8.14) ...
Setting up libgtk-4-1:amd64 (4.14.5+ds-0ubuntu0.9) ...
Setting up libgtk-4-bin (4.14.5+ds-0ubuntu0.9) ...
Setting up gir1.2-gdm-1.0 (46.2-1ubuntu1~24.04.7) ...
Setting up gir1.2-gtk-4.0:amd64 (4.14.5+ds-0ubuntu0.9) ...
Setting up network-manager-openvpn-gnome (1.10.2-4ubuntu0.2) ...
Setting up libgtk-4-media-gstreamer (4.14.5+ds-0ubuntu0.9) ...
Setting up gdm3 (46.2-1ubuntu1~24.04.7) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
shraddha-sinha@shraddha-sinha-VirtualBox:~$
```

Part C — The FOSS Ecosystem (Units 3 & 4)

1. Dependencies (Libraries, Compilers, Runtimes)

Python needs free and open-source parts to work well. It is written in C so it needs a C compiler like GCC to build and develop. It also uses system libraries like C libraries from Linux. Python comes with a standard library and supports many third-party libraries like NumPy, Pandas and TensorFlow which make it more useful. The pip package manager helps install and manage these libraries. These dependencies make Python strong and flexible for uses.

2. What Has It Enabled or Inspired?

Python has helped create tools, frameworks and applications. Popular frameworks like Django and Flask for building websites are made with Python. In data science and artificial intelligence tools like TensorFlow, PyTorch and Scikit-learn are widely. Rely on Python. Pythons simplicity and readability have even influenced programming languages. It has enabled development and innovation in areas like automation, machine learning and software development with Python.

3. Role in the Software Ecosystem (Relation to Web / LAMP Stack)

Python is important in todays software world for building websites. Even though its not part of the LAMP stack (Linux, Apache, MySQL, PHP) Python can replace PHP and be used with frameworks like Django or Flask to build dynamic websites with Python. It works well with databases like MySQL and PostgreSQL. Can run on web servers like Apache or Nginx with Python. Python is also used for backend development, APIs and cloud-based applications, with Python. Its flexibility makes Python a key part of todays software ecosystem.

Part D — Open Source vs Proprietary (Critical Analysis)

Software can be put into two groups: open-source software and proprietary software. Open-source software lets people see its code so they can change it. Share it for free. On the hand proprietary software keeps its code secret and is controlled by one company. Both types have bad points and we can see how they affect things like new ideas, security, cost and control in different areas

Dimension	Open Source	Proprietary Alternative
Cost	Free to use	Often paid or licensed
Security (who can audit the code?)	Can be audited by anyone	Depends on company
Support and reliability	Fast (Distributed System)	Slower (centralised system)
Freedom to modify	Highly customizable	Limited customization
Community vs corporate control	Community-driven	Company-controlled

Your overall verdict	Overall better	Less better

1. Transparency and Control
One big advantage of open-source software is that it is transparent. Since everyone can see the code users and developers can figure out how the software works, find mistakes and even change it to suit their needs. This gives users control over the open-source software.
Proprietary software is different. It does not let people see its code. Users have to trust the company that makes it. They have limited control over how it works. Any changes or improvements depend on the company that makes the software.

2. Cost and Accessibility

Open-source software is usually free. It is easy for students, developers and organizations with small budgets to use. It removes money problems. Encourages lots of people to use it.
On the hand proprietary software often requires people to buy licenses or subscriptions. This can be expensive. It usually comes with help from the company and regular updates.

3. Security and Reliability
Open-source software is often seen as secure because many developers around the world can look at it and fix problems quickly. This team approach leads to discovery of issues with the open-source software.
However proprietary software can also be secure. Companies spend a lot of money on testing and security measures for their software. The difference is that problems with software are handled inside the company and are not visible to the public.

4. Innovation and Development
Open-source software helps ideas by letting developers build on what already exists. It encourages people to work and share knowledge, which speeds up technological progress in the open-source software field.
Proprietary software however often focuses on development. Companies spend money on research to create products. New ideas happen in both types. They are done differently in open-source software and proprietary software.

5. Support and Maintenance
Open-source software relies on help from the community, such as forums and people who contribute. While this can be very helpful it may not always give answers to questions about the open-source software.
Proprietary software usually offers help regular updates and maintenance. This makes it more reliable for businesses that need support for their proprietary software.

Conclusion
Both open-source software and proprietary software have bad points. Open-source software gives people freedom, flexibility. Helps new ideas while proprietary software offers stability, dedicated help and controlled development.
In my opinion open-source software is better for learning, research and new ideas while proprietary software is better for businesses that need reliability and support. Using both open-source software and proprietary software in a way can be the most effective approach, in real life.

4. Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

Write a script that introduces the Linux system like a welcome screen. It should display:

- The Linux distribution name and kernel version
- The current logged-in user and their home directory
- System uptime and current date/time
- A message stating which open-source license covers the OS

Concepts to use: variables, echo, command substitution ($()), basic output formatting.

Solution: I am working on a shell script. A shell script is a program that automates the execution of commands in a Linux environment. I used a language to write this program.

I used variables to store the output of some commands. These commands are whoami, uname -a python3 --version and which python3. I used command substitution to assign the output of these commands to the variables. The command substitution is done using $().

I used the echo command to display the results. I made the results easy to read by using separators and spacing. This makes the output easy to understand.

I saved the script with a.sh extension. Then I gave it execution permission using the chmod +x command. After that I executed the script using./myscript.sh. This shell script shows how shell scripting can be used to automate tasks. It also shows how to display system information in a way. The shell script is a tool for automating tasks and displaying system information, like the output of whoami uname -a python3 --version and which python3.

Screenshots of the steps and output:

```
shraddha-sinha@shraddha-sinha-VirtualBox: ~
  GNU nano 7.2                                   myscript.sh *
echo "   SYSTEM INFORMATION SCRIPT   "
echo "==============================="

echo ""
echo "User: $current_user"
echo ""
echo "System Info: $system_info"
echo ""
echo "Python Version: $python_version"
echo ""
echo "Python Location: $python_path"
echo ""
echo "==============================="
echo "Script Execution Completed"

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

```
shraddha-sinha@shraddha-sinha-VirtualBox: ~
shraddha-sinha@shraddha-sinha-VirtualBox:~$ nano myscript.sh
shraddha-sinha@shraddha-sinha-VirtualBox:~$ chmod +x myscript.sh
shraddha-sinha@shraddha-sinha-VirtualBox:~$ ./myscript.sh
===============================
   SYSTEM INFORMATION SCRIPT
===============================

User: shraddha-sinha

System Info: Linux shraddha-sinha-VirtualBox 6.17.0-19-generic #19~24.04.2-Ubuntu SMP PREEMPT_DYNAMIC Fri Mar  6 23:08:4
6 UTC 2 x86_64 x86_64 x86_64 GNU/Linux

Python Version: Python 3.12.3

Python Location: /usr/bin/python3

===============================
Script Execution Completed
shraddha-sinha@shraddha-sinha-VirtualBox:~$
```

Script 2 — FOSS Package Inspector (Unit 2)

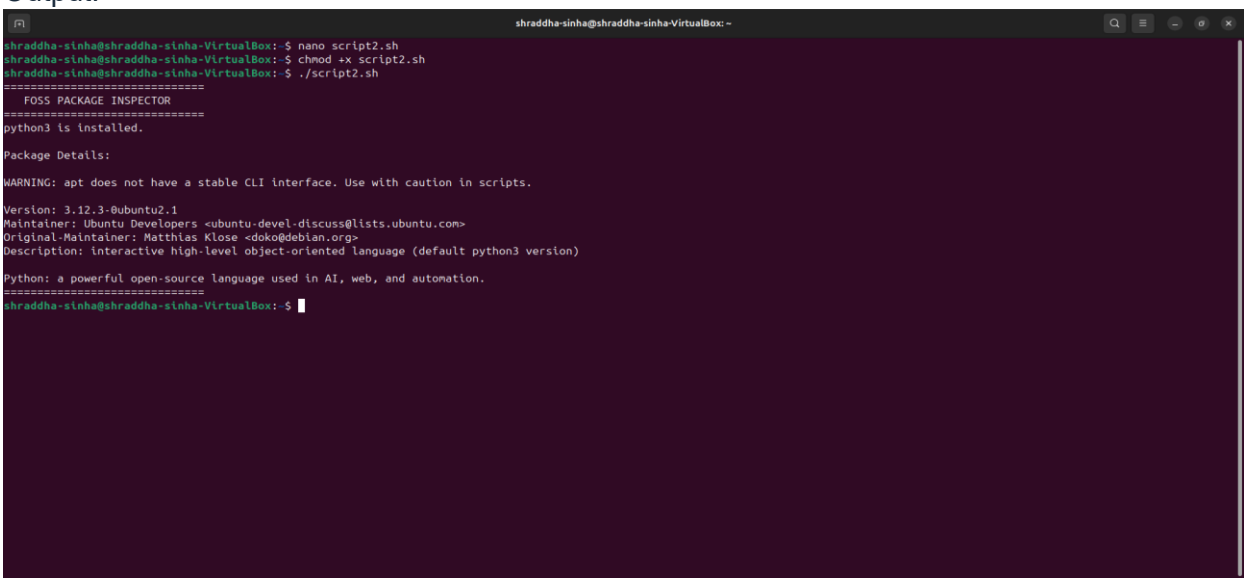
This script checks if a specific open-source package is installed on a Linux system.
The package we are checking for is Python.
It uses an if- statement to see if Python is installed.
This is done using the dpkg command.
If Python is installed the script shows details like its version and description.
The apt show command is used for this.
The script also uses a case statement.
This displays a description of Python based on its name.
It shows how to use statements in shell scripting.
These statements help handle situations.
The script is useful for understanding how Linux handles software packages.
It also shows how to get information, about installed software.
The script uses Python as an example to demonstrate this.
Python is an open-source package.
It is often used on Linux systems.

Screenshots of the process and output:

```
shraddha-sinha@shraddha-sinha-VirtualBox: ~
  GNU nano 7.2                                   script2.sh *
#!/bin/bash

# Package name
PACKAGE="python3"

echo "============================="
echo "   FOSS PACKAGE INSPECTOR"
echo "============================="

# Check if package is installed
if dpkg -l | grep -q $PACKAGE; then
    echo "$PACKAGE is installed."

    echo ""
    echo "Package Details:"
    apt show $PACKAGE | grep -E 'Version|Maintainer|Description'
else
    echo "$PACKAGE is NOT installed."
fi

echo ""

# Case statement for description
case $PACKAGE in
    python3)
        echo "Python: a powerful open-source language used in AI, web, and automation."
        ;;
    git)
        echo "Git: version control system for tracking code changes."
        ;;
    vlc)
        echo "VLC: open-source media player supporting all formats."
        ;;
    apache2)
        echo "Apache: web server that powers many websites."

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-] To Bracket  M-Q Previous  ^B Back
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy      ^Q Where Was    M-W Next      ^F Forward
```

```
shraddha-sinha@shraddha-sinha-VirtualBox: ~
  GNU nano 7.2                                   script2.sh *
# Check if package is installed
if dpkg -l | grep -q $PACKAGE; then
    echo "$PACKAGE is installed."

    echo ""
    echo "Package Details:"
    apt show $PACKAGE | grep -E 'Version|Maintainer|Description'
else
    echo "$PACKAGE is NOT installed."
fi

echo ""

# Case statement for description
case $PACKAGE in
    python3)
        echo "Python: a powerful open-source language used in AI, web, and automation."
        ;;
    git)
        echo "Git: version control system for tracking code changes."
        ;;
    vlc)
        echo "VLC: open-source media player supporting all formats."
        ;;
    apache2)
        echo "Apache: web server that powers many websites."
        ;;
    *)
        echo "Unknown package."
        ;;
esac

echo "============================="

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-] To Bracket  M-Q Previous  ^B Back
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy      ^Q Where Was    M-W Next      ^F Forward
```

Output:



Script 3 — Disk and Permission Auditor (Unit 2)

This script helps analyze Linux system directories.

It checks directories like /etc /var/log, /home /usr/bin and /tmp.

* It loops through each directory.

* For each one it checks if the directory exists.

If it does the script gets the directorys permissions. Who owns it.

It uses the ls -ld command for this.

The script also calculates the directory size with du -sh.

The output is made clear with echo statements.

This script is about using loops and commands to get system info.

It helps you understand file permissions, the directory structure and disk usage in Linux systems.

The script uses a for loop to go through the directories.

It uses an if statement to check if each directory exists.

The script shows how to use Linux commands like ls and du.

These commands help get info about directories.

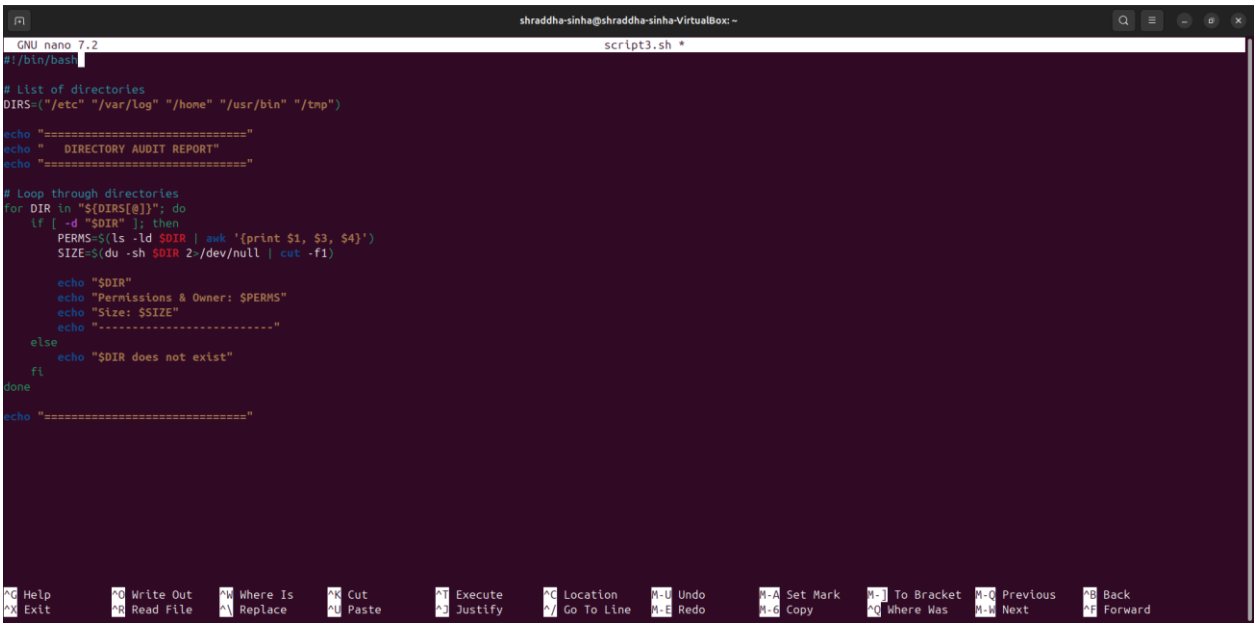
The script is useful for learning about Linux system directories.

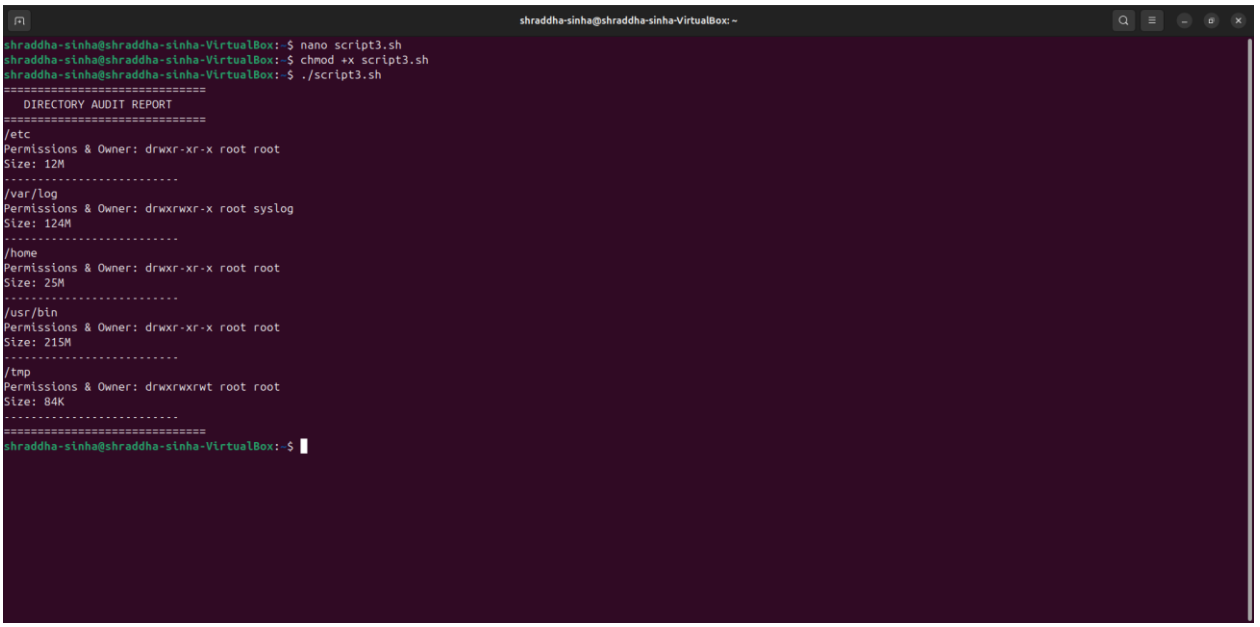
It helps with understanding how to get info about directories.

The directories are like /etc. /Log.

They are important, for the Linux system.

Screenshots of process and outputs:





Script 4 — Log File Analyzer (Units 2 & 5)

This script looks at a log file on a Linux system. It counts how times a certain word, like "error" or "warning" appears in the file.

You give the script the name of the log file and the word you want to look for.

If you do not tell it what word to look for it will look for "error" by default.

The script reads the file line by line.

It checks each line to see if it has the word you are looking for.

It uses a counter to keep track of how many lines have the word.

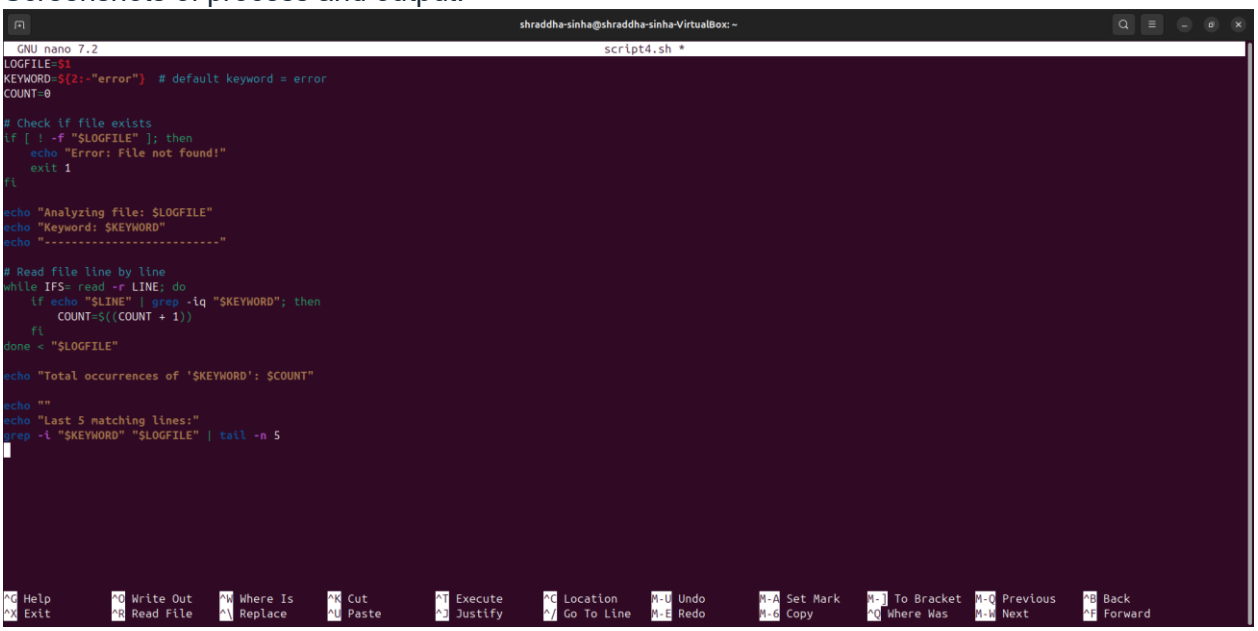
When it finishes it tells you how times the word appeared.

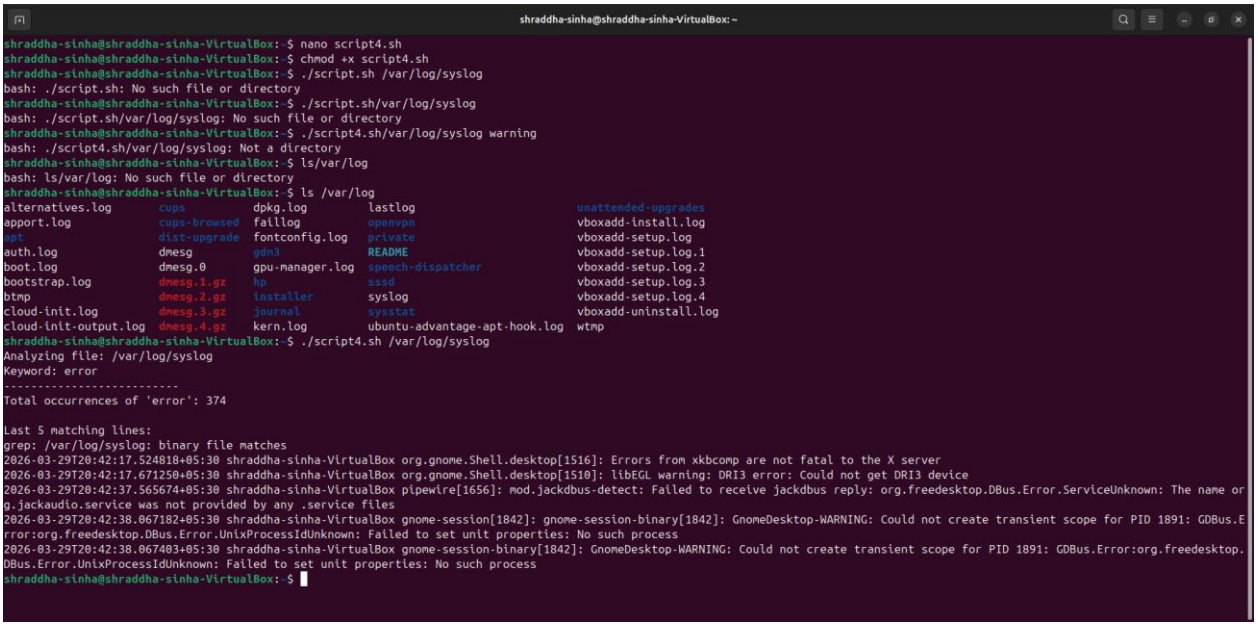
It also shows you the five lines that had the word.

This script is an example of how to work with files use loops and conditions and handle inputs in shell scripting.

It uses commands, like grep and tail to get the information it needs.

Screenshots of process and output:

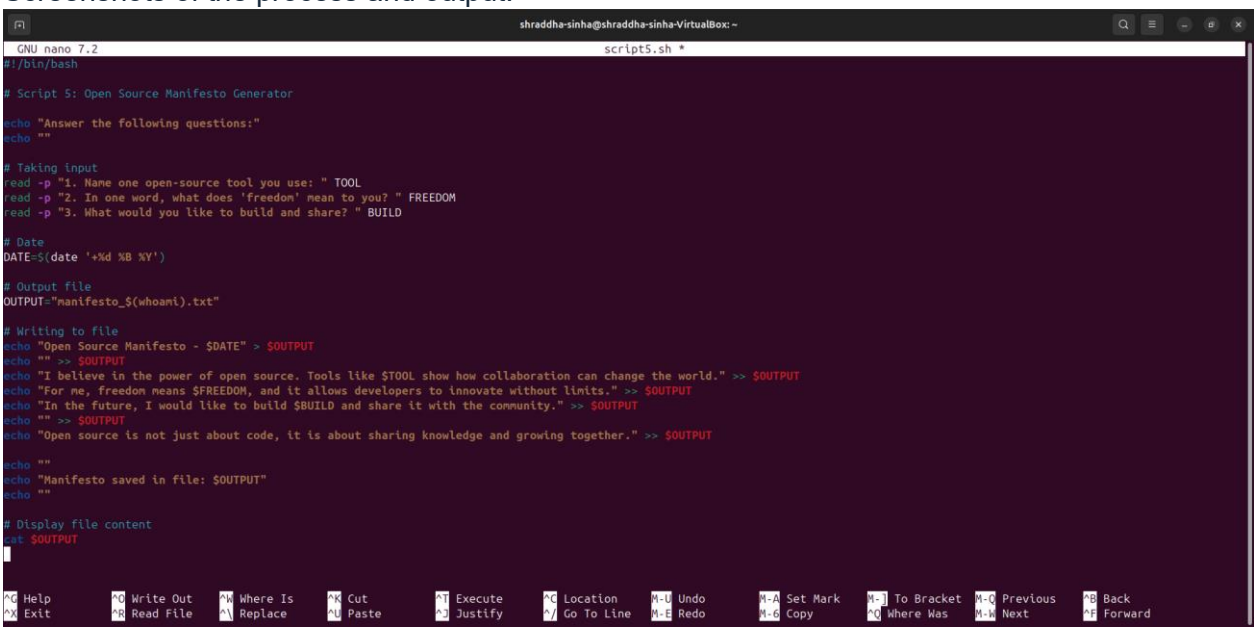
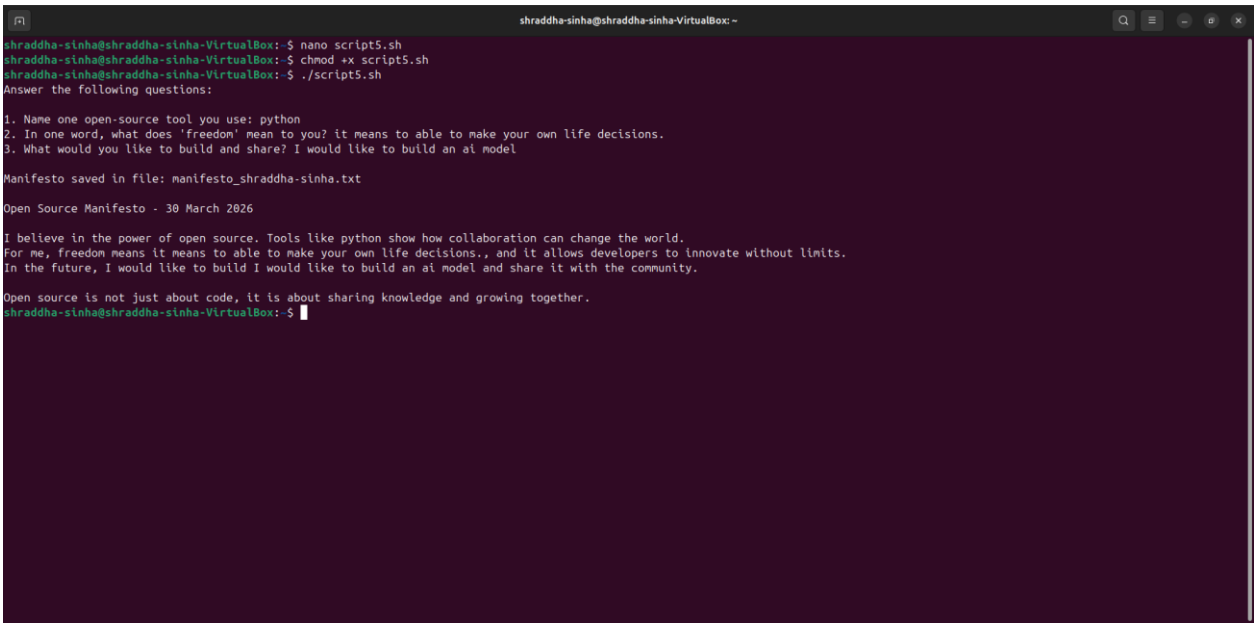




Script 5 — The Open Source Manifesto Generator (Unit 5)

This script makes an open-source manifesto just for you, based on what you tell it. It asks you for things like the name of an open-source tool you like what freedom means to you and something you want to create and share with others.

The script remembers what you say by storing it in variables. It also keeps track of the current date. Then it makes a text file and writes a paragraph that puts all your ideas together. The manifesto that is made shows what you think about open-source philosophy. It is all about open-source tools and what they mean to you.

The script then shows you what is in the file so you can read your open-source manifesto. This script is an example of how you can talk to the computer work with files and play with words in shell scripting and it is all, about open-source tools and how they work.

Screenshots of the process and output: